

Handwritten Digit Recognition Using Backpropagation Neural Network

Submitted By: Hassan Raza

Roll Number: BSE-F23-M49

Subject: Artificial Intelligence

Submitted To: Mr. Asif Raza

University: University of Mianwali

Abstract:

Artificial Intelligence and Machine Learning are growing fast, changing how computers solve problems every day. Image recognition is a key application in this field. In this project, I built a Handwritten Digit Recognition System using a Backpropagation Neural Network. The main goal? To teach a model to recognize handwritten digits from 0 to 9 by training it on real-world data.

I used the Scikit-learn digits dataset and wrote the neural network in Python with NumPy. No fancy deep learning libraries—just the basics. The system runs forward propagation, backpropagation, updates weights as it learns, makes predictions, and tests its accuracy.

In the end, the model reached about 95% accuracy. From preprocessing the data, to training and testing, to visualizing the learning process with graphs, all stages came together well. The results show that even simple neural networks can tackle image recognition tasks with impressive accuracy.

Keywords: Artificial Intelligence, Neural Network, Backpropagation, Handwritten Digit Recognition, Machine Learning, Python

1. Introduction:

Artificial Intelligence is one of the driving forces in computer science right now. AI systems are all about learning patterns in data and making smart decisions. Machine Learning, a big part of AI, lets computers learn automatically from experience—no hardcoded rules needed.

Image recognition pops up everywhere these days. Recognizing handwritten digits, for example, is a classic problem in computer vision. You'll find it in banking, mail sorting, automated form readers, education, and more.

For this project, I used a Backpropagation Neural Network to identify handwritten digits. You show it an image, and it predicts the right digit—anything from 0 to 9. The neural network trains on sample images, tweaks itself to get better, and reduces its mistakes as it goes. Python and libraries like NumPy, Matplotlib, and Scikit-learn made the job smoother.

2. Problem Statement:

Traditional computers really struggle with handwritten digits; everyone writes differently. Shapes and sizes of the same number change from person to person. That's why you need a learning system—something smart enough to spot these variations and recognize digits accurately.

The main challenge here:

Build a neural network model that can recognize handwritten digits using backpropagation.

3. Objectives:

Here's what I set out to do:

- Understand how Artificial Neural Networks work.
- Implement Forward Propagation and Backpropagation from scratch.
- Train a neural network using handwritten digit data.
- Classify digits from 0 to 9 correctly.
- Check the model's accuracy.
- Visualize the learning process with graphs.

4. Literature Review:

Handwritten digit recognition has kept researchers busy in AI and computer vision for decades. People have tried everything—from SVMs and Decision Trees to KNN and Neural Networks.

Neural networks especially shine at pattern recognition, picking up on complex relationships in data. Backpropagation is probably the most-used way to train them: you figure out where you went wrong, send that error backward through the network, and adjust the weights to do better next time.

In the last few years, Deep Learning has taken things to the next level. Still, simpler neural networks give you a solid understanding of how machines learn from data.

5. Methodology:

Here's how the project unfolded, step by step:

1. Collect the dataset.
2. Preprocess the data.
3. Design the neural network.
4. Run forward propagation.
5. Run backpropagation.
6. Train the model.
7. Predict and test with new images.
8. Check accuracy.

9. Make graphs for visualization.

6. Dataset Description:

I worked with the Scikit-learn handwritten digits dataset.

Quick stats:

- Total images: 1797
- Each image: 8×8 pixels (so, 64 features)
- Digits covered: 0 to 9

Images use grayscale values for each pixel.

Loading the data was easy:

```
from sklearn.datasets import load_digits  
digits = load_digits()
```

7. Data Preprocessing:

You can't just feed raw data to a machine learning model. It needs a bit of help first. Here's what I did:

7.1 Data Normalization

Pixel values were scaled between 0 and 1:

$X = X / 16.0$

This speeds up training and helps the model learn better.

7.2 One-Hot Encoding

Labels (the correct digits) turned into vectors:

Digit 3 becomes [0 0 0 1 0 0 0 0 0].

7.3 Train-Test Split

Split the dataset:

- 80% for training
- 20% for testing

This way, the model learns from most of the data and we can see how it does on new images.

8. Neural Network Architecture:

The network has three layers:

- Input Layer: 64 neurons (one per pixel)
- Hidden Layer: 128 neurons, uses ReLU activation
- Output Layer: 10 neurons (one per digit), uses Sigmoid activation

9. Forward Propagation:

Here's how data moves through the network:

- Hidden Layer:

$$Z1 = XW1 + b1$$

$$A1 = \text{ReLU}(Z1)$$

- Output Layer:

$$Z2 = A1W2 + b2$$

$$A2 = \text{Sigmoid}(Z2)$$

In the end, you get a set of probabilities for each digit.

10. Backpropagation:

Backpropagation helps the network learn from its mistakes:

1. Calculate how wrong the prediction was.
2. Push that error backwards through the layers.
3. Adjust weights and biases.
4. Try to lower the loss (the amount it got wrong).

Update formula:

$$W = W - \eta(dL/dW)$$

Where W is the weight, η is the learning rate, and dL/dW is the loss gradient.

After each training round (epoch), the model gets a little better.

11. Activation Functions:

11.1 ReLU (Rectified Linear Unit)

Used in the hidden layer:

$$\text{ReLU}(x) = \max(0, x)$$

Why? It's fast, efficient, and helps fix the vanishing gradient problem.

11.2 Sigmoid

Used in the output layer:

$$\text{Sigmoid}(x) = 1 / (1 + e^{-x})$$

Perfect for outputting probabilities between 0 and 1.

12. Model Training:

I trained the network for 1000 epochs. Here's what that looked like:

- Epochs: 1000
- Learning Rate: 0.1
- Hidden Neurons: 128

With each round:

- The model made a prediction.
- Calculated the loss.
- Ran backpropagation.
- Improved its weights.

Loss steadily went down—proof the model was learning.

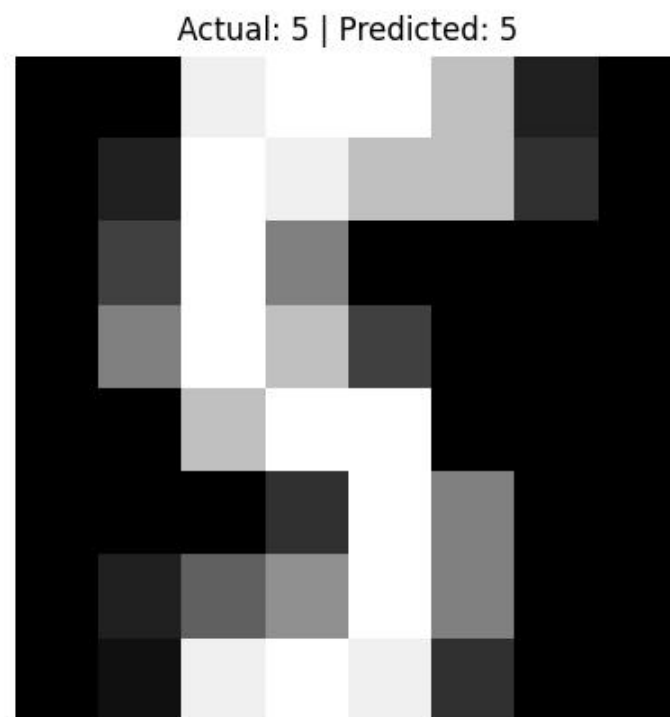
13. Results and Discussion:

The big question: did it work? Yes—it recognized digits with about 95.83% accuracy. That's a strong result, especially for a simple neural network.

The loss graph showed a steady drop as training went on. The model made accurate predictions on test images. It's clear that Backpropagation Neural Networks work well for image classification.

14. Figures:

Figure 1: Handwritten Digit Prediction



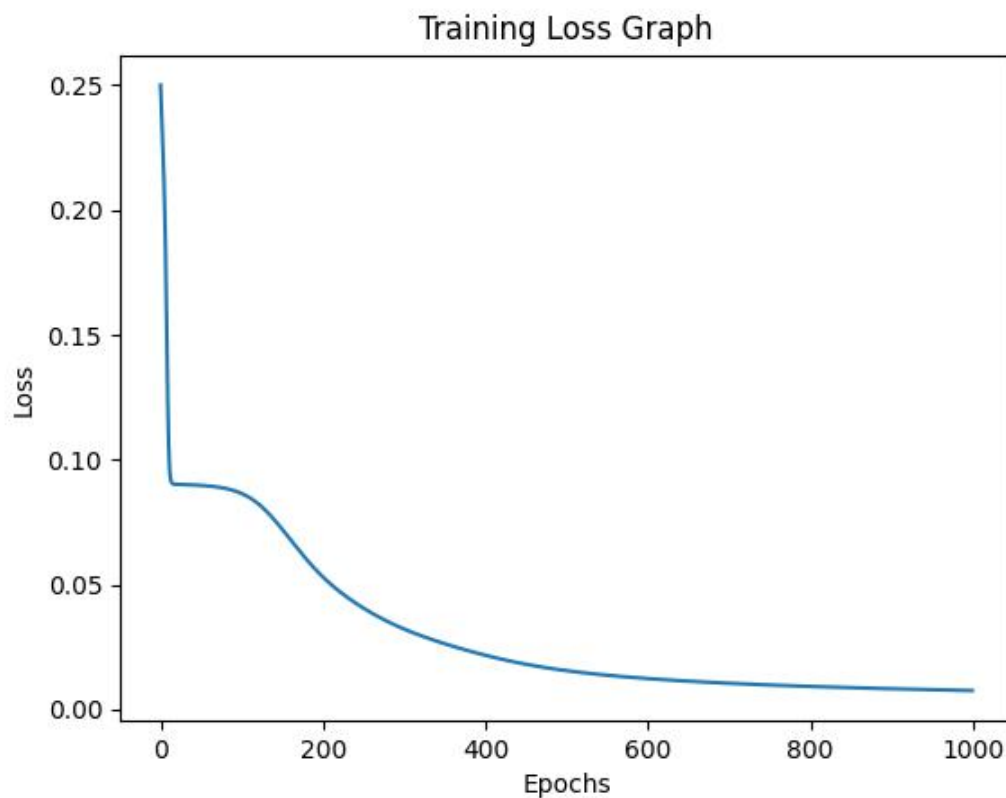
The model guessed the digit correctly.

Example:

Actual Digit: 5

Predicted Digit: 5

Figure 2: Training Loss Graph



Loss dropped steadily across training—proof of good learning.

Figure 3: Dataset Information

```
Training Data Shape:
(1437, 64)

Testing Data Shape:
(360, 64)

Training Labels Shape:
(1437, 10)

Testing Labels Shape:
(360, 10)
```

Showed the size of training and test sets.

Figure 4: Model Accuracy

```
Model Accuracy:
95.83333333333334 %
```

Final accuracy: about 95.83%.

15. Advantages of the System

- High prediction accuracy
- Learns automatically from data

- Fast and efficient at recognizing digits
- Can make predictions in real time
- Builds a solid understanding of neural networks

16. Limitations:

- Training takes longer on bigger datasets
- Basic networks fall short compared to advanced deep learning models
- Accuracy drops if images are noisy
- This model is only trained on (8 * 8) pixels. So if we give manually any image it will not predict the correct digit. But it works good on MNIST dataset. If we want that the model should work on both manual digit image we can train it on (28 * 28) pixels.

17. Future Work:

Here's where the project could go next:

- Try Convolutional Neural Networks (CNNs)
- Use a bigger dataset
- Make a friendlier interface
- Improve optimization algorithms
- Build a web app for deployment

18. Conclusion:

This project showed how a Backpropagation Neural Network can recognize handwritten digits with high accuracy. It covered everything from understanding neural networks and training methods to handling data and measuring results.

With an accuracy of around 95%, it's clear that neural networks can handle image recognition tasks very well. This project is a good example of AI in action, solving real-world classification problems.

19. References:

1. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, Deep Learning, MIT Press
2. Scikit-learn Documentation: <https://scikit-learn.org/>
3. NumPy Documentation: <https://numpy.org/>
4. Matplotlib Documentation: <https://matplotlib.org/>
5. Simon Haykin, Neural Networks and Learning Machines
6. Python Official Documentation: <https://www.python.org/>

Appendix:

Tools and Technologies Used

- Python

- NumPy
- Matplotlib
- Scikit-learn
- VS Code
- Anaconda Navigator

Project Files:

- main.py
- neural_network.py
- train.py
- predict.py
- utils.py
- requirements.txt

End of Report